PROJECT PRESENTATION | TEAM 11

Smart Warehouse AGV Dispatcher

Intelligent Multi-Robot Task Allocation and Pathfinding

SUBMITTED BY

Team 11

INSTITUTE

IIT Patna

Python | Pygame | A* Search | Hungarian Algorithm

TEAM MEMBERS

Submitted by: Team 11

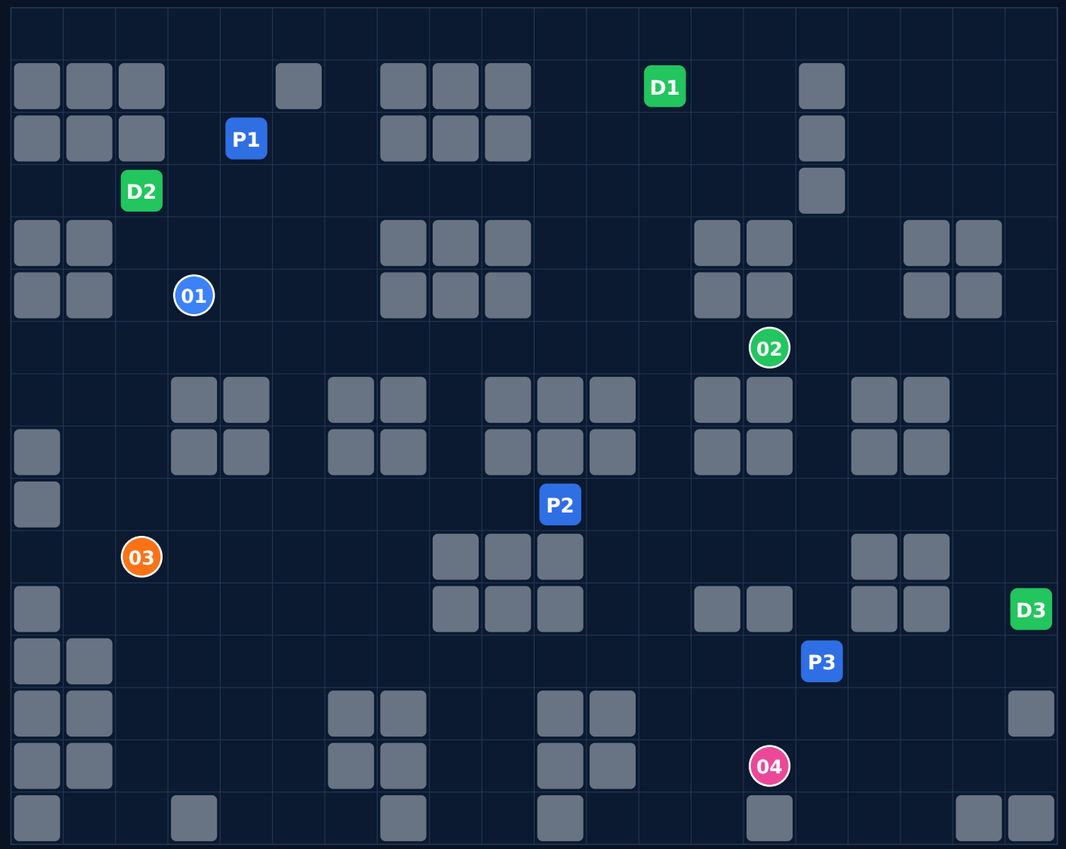
Ratan Kumar Mahato	2501AI18
Anubhav Kumar	2501AI05
Jitesh Kumar	2501AI29
Devansh Yadav	2501AI25
Devesh Yadav	2501AI38
Aman Raj	2501AI46
Chitra Sandilya	2501CS20
Riya Raj	2501AI22
Aniska Saha	2503CS04

01 PROBLEM

Warehouse Automation

AGVs carry packages from pickup points to delivery stations.
With a fleet, every new order raises the same two questions.

1. **Different robots**
   speed 1.5-3.0 cells/s, capacity 60-150 kg, energy 0.4-0.8 per cell
2. **Continuous orders**
   they arrive over time; some are too heavy for most robots
3. **Shared floor**
   one grid with shelves; two robots must never share a cell
4. **Competing goals**
   fast, cheap and evenly shared work pull in different directions



The built-in warehouse and its four AGVs

Which robot takes which order, and which way does it drive?

Problem Statement and Objectives

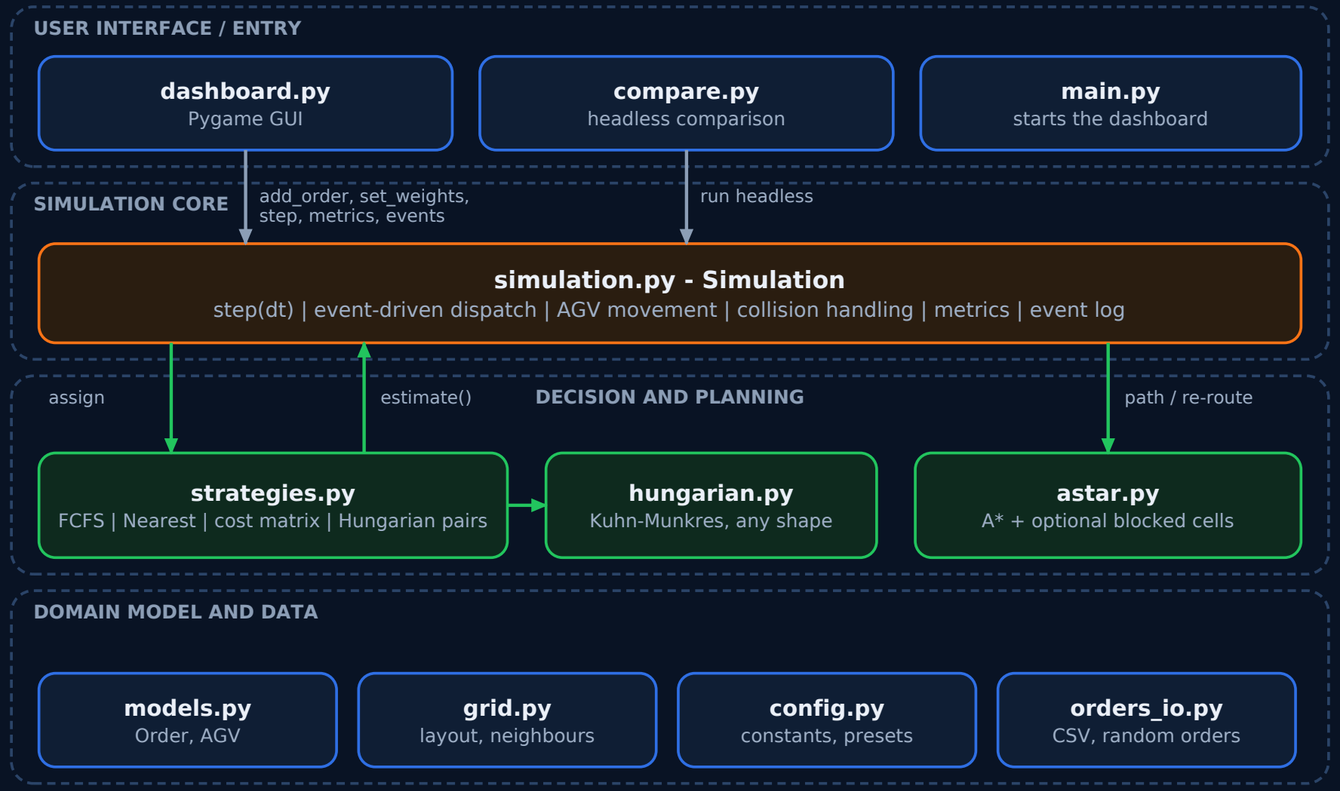
Given a warehouse grid, AGVs with different speeds, capacities and energy rates, and a stream of orders: decide who carries what and along which route - quickly, with little empty travel and energy, with shared workload, and without two robots in one cell.

1. **Heterogeneous fleet**
   own speed, load limit, energy rate
2. **Shortest routes**
   A* on the grid
3. **Best batch assignment**
   Hungarian algorithm, rectangular cases
4. **Adjustable cost**
   time, energy and workload weights
5. **Safe movement**
   reservation, waiting, re-route, back-off
6. **Fair evaluation**
   same orders, real measured metrics

One system that allocates, routes, coordinates and measures.

Proposed System

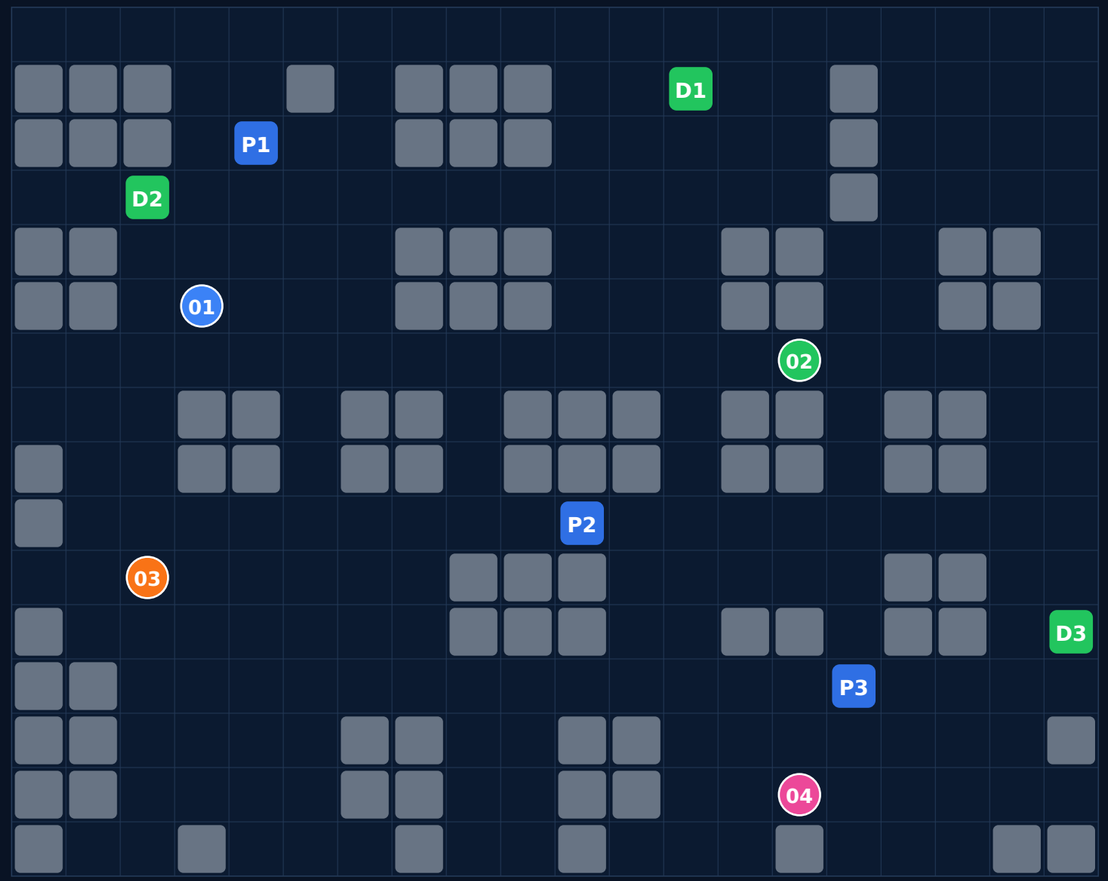




A* and Hungarian are separate modules; the simulation is the core.

04 ENVIRONMENT

Warehouse Environment and AGVs



20 x 16 grid | shelves, 3 pickups, 3 deliveries | 4-way moves

AGV	Speed	Capacity	Energy/cell	Rs/s
AGV-01	2.0 cells/s	100 kg	0.5	0.30
AGV-02	3.0 cells/s	60 kg	0.8	0.45
AGV-03	1.5 cells/s	150 kg	0.4	0.35
AGV-04	2.5 cells/s	80 kg	0.6	0.40

Energy per cell rises by up to 30% with load.

Every order names a pickup, a delivery and a weight. A package heavier than a robot's capacity is never given to that robot - an order heavier than all of them fails.

20 x 16
grid cells

93
shelf cells

3 + 3
pickups + deliveries

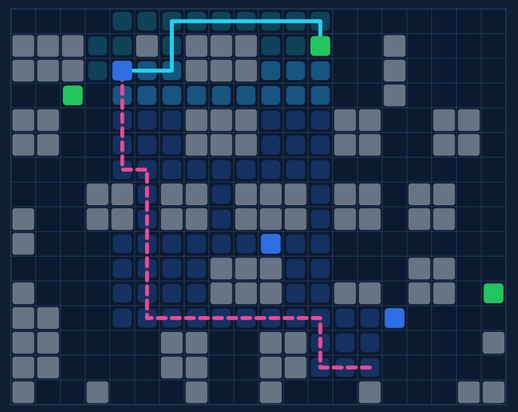
Four different robots on one shared grid: the fastest is the lightest, the strongest is the slowest.

05 ALGORITHM

A* Pathfinding

$$f(n) = g(n) + h(n), \qquad h(n) = |x_n - x_{goal}| + |y_n - y_{goal}|$$

- g(n): steps walked so far; h(n): Manhattan distance to the goal
- Moves are 4-directional with cost 1, so h never over-estimates: the first route found is shortest
- Optional blocked cells let the same A* route around other AGVs

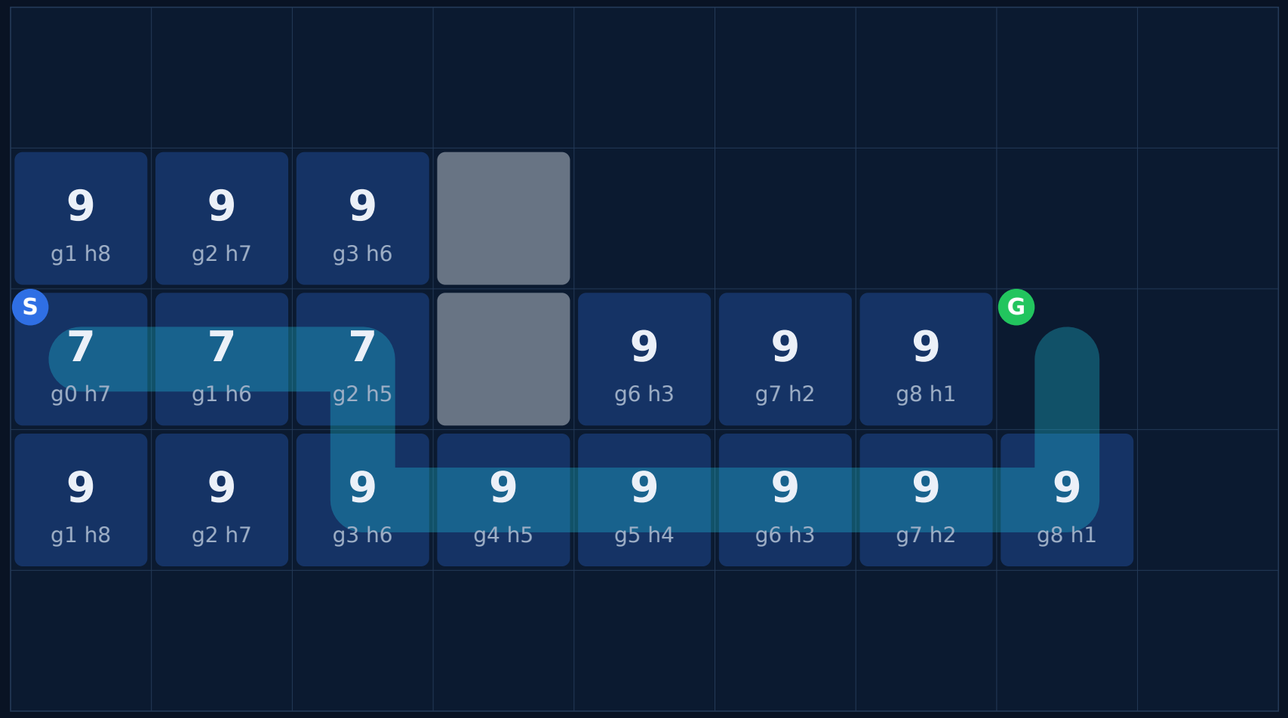


Real route, built-in warehouse

AGV-04 to P1: 22 cells (the Manhattan bound), 79 cells examined.

P1 to D1: 11 cells, 30 examined.

Dashed = empty leg, solid = loaded leg, shaded = examined.



Number in a cell = f = g + h | small text = g (steps so far) and h (Manhattan distance to G) | 17 cells expanded, route 9 steps

Teaching example (real run of the project's A*): a wall forces a detour. Cells on the way show f = g + h; f stays at 7 until the wall, then 9 = the optimal route length.

A* looks toward the goal and returns a shortest route; the same code re-routes around other robots.

Hungarian Algorithm

$$\min \sum_i \sum_j c_{ij}\, x_{ij} \quad \text{s.t.} \quad \sum_j x_{ij} \le 1, \quad \sum_i x_{ij} \le 1, \quad x_{ij} \in \{0, 1\}$$

- Rows = pending orders, columns = AGVs with room
- Impossible pair (too heavy / no route) costs 1,000,000
- Rectangular is fine: min(orders, AGVs) pairs, the rest wait
- Exact optimum, O(n²m): no trial of all n! assignments
- Checked against brute force on 200 random matrices
- Runs again whenever the situation changes

Cost matrix C(r, o): real values from the project

	AGV-01	AGV-02	AGV-03	AGV-04
#1001 P1 > D2 30 kg	**0.279**	0.512	0.431	0.631
#1002 P2 > D1 50 kg	0.565	0.538	0.570	**0.530**
#1003 P3 > D3 110 kg	n/a	n/a	**0.593**	n/a
#1004 P2 > D3 20 kg	0.578	**0.541**	0.588	0.539

Total cost

Hungarian 1.943

FCFS / Nearest 1.949

#1003 (110 kg) can only go to AGV-03. Hungarian swaps #1002 and #1004: a small gain that shows the mechanism.

Highlighted = Hungarian choice, total cost 1.943. FCFS / Nearest (same picks) total 1.949. n/a = order too heavy for that AGV.

Hungarian finds the cheapest set of pairs; impossible pairs are excluded, not forced.

07 ALLOCATION

Task Allocation Strategies

FCFS

Oldest order first to the AGV that is free soonest.

USES

Ignores distance, speed, energy.

WEAKNESS

Can send a far robot to a near pickup.

Nearest AGV

Oldest order first to the AGV closest to the pickup.

USES

Distance only.

WEAKNESS

A near slow robot can lose to a far fast one.

Hungarian

All pending orders and free robots solved together on a weighted cost.

USES

Speed-aware T, energy cost E, workload W.

WEAKNESS

Ignores waiting for other robots.

Who gets which order? (real run, four orders at time 0)

	FCFS	Nearest	Hungarian
#1001 P1 > D2 30 kg	AGV-01	AGV-01	AGV-01
#1002 P2 > D1 50 kg	AGV-02	AGV-02	AGV-04
#1003 P3 > D3 110 kg	AGV-03	AGV-03	AGV-03
#1004 P2 > D3 20 kg	AGV-04	AGV-04	AGV-02

Same four orders at time 0. Outlined cells: Hungarian gives a different AGV than FCFS / Nearest (it swaps #1002 and #1004).

What differs

FCFS and Nearest give the same result here. Hungarian gives #1002 to AGV-04 and #1004 to AGV-02, because that lowers the total cost.

Same orders, different rules: Hungarian can swap assignments that greedy rules keep.

Optimization Objectives

$$C(r,o) = \alpha \hat{T}(r,o) + \beta \hat{E}(r,o) + \gamma \hat{W}(r)$$

T Time

(d1 + d2) / speed + loading

own speed; includes the empty leg

E Energy cost

energy x price + T x time cost

loaded legs cost more per cell

W Workload

seconds of work already owned

spreads orders over the fleet

Presets (alpha / beta / gamma)

Time	0.7 / 0.1 / 0.2
Energy	0.1 / 0.7 / 0.2
Balanced	0.4 / 0.3 / 0.3

Same 30 kg order, two robots on the same cell



Winner for each preset

Time 0.7 / 0.1 / 0.2	**Fast**
Energy 0.1 / 0.7 / 0.2	**Thrifty**
Balanced 0.4 / 0.3 / 0.3	**Thrifty**

Changing the weights changes the decision: time picks Fast, energy picks Thrifty.

09 COORDINATION

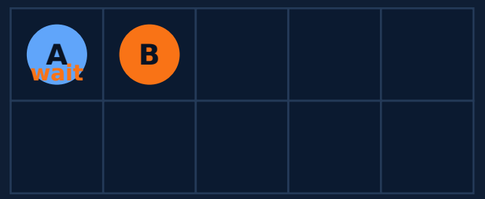
Collision Avoidance and Deadlock Handling

1 Reserve



Reserve the next cell before entering. One owner per cell.

2 Wait



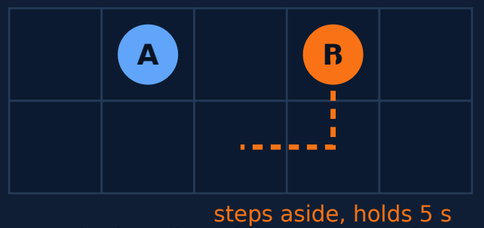
Cell taken? Stay put. Idle robots in the way step aside.

3 Re-route (4 s)



Blocked 4 s: A* again, avoiding occupied cells.

4 Back off (8 s)



In a circle of waiting AGVs the lowest-ranked steps aside, holds 5 s.

Verified by tests after every step

- never two AGVs in one cell, never one cell reserved twice
- never two AGVs swapping cells
- 12 runs x 40 orders finished without deadlock

Real event log (benchmark run, seed 1)

```
AGV-03 is waiting for AGV-02
AGV-02 is waiting for AGV-03
AGV-03 found another route
AGV-03 is waiting for AGV-04
```

Simple rules, not planning: reserve, wait, re-route, back off.

10 WORKFLOW

Order Management & Dynamic Assignment

01 ORDER INPUT

Manual entry

Enter pickup, delivery and package weight through the dashboard.

02 BULK GENERATION

Random orders

Generate multiple valid orders at once to test changing workloads.

03 CSV IMPORT / EXPORT

Batch order management

Load orders with pickup, delivery, weight and arrival time. Invalid rows are skipped and reported.

EVENT-DRIVEN DISPATCH CYCLE

1 New event

Order arrives, delivery completes, or settings change.

2 Update queue

Only waiting orders return to the pending queue.

3 Validate

Reject orders no AGV can carry or reach.

4 Assign

Hungarian algorithm matches pending orders to available AGVs.

5 Execute

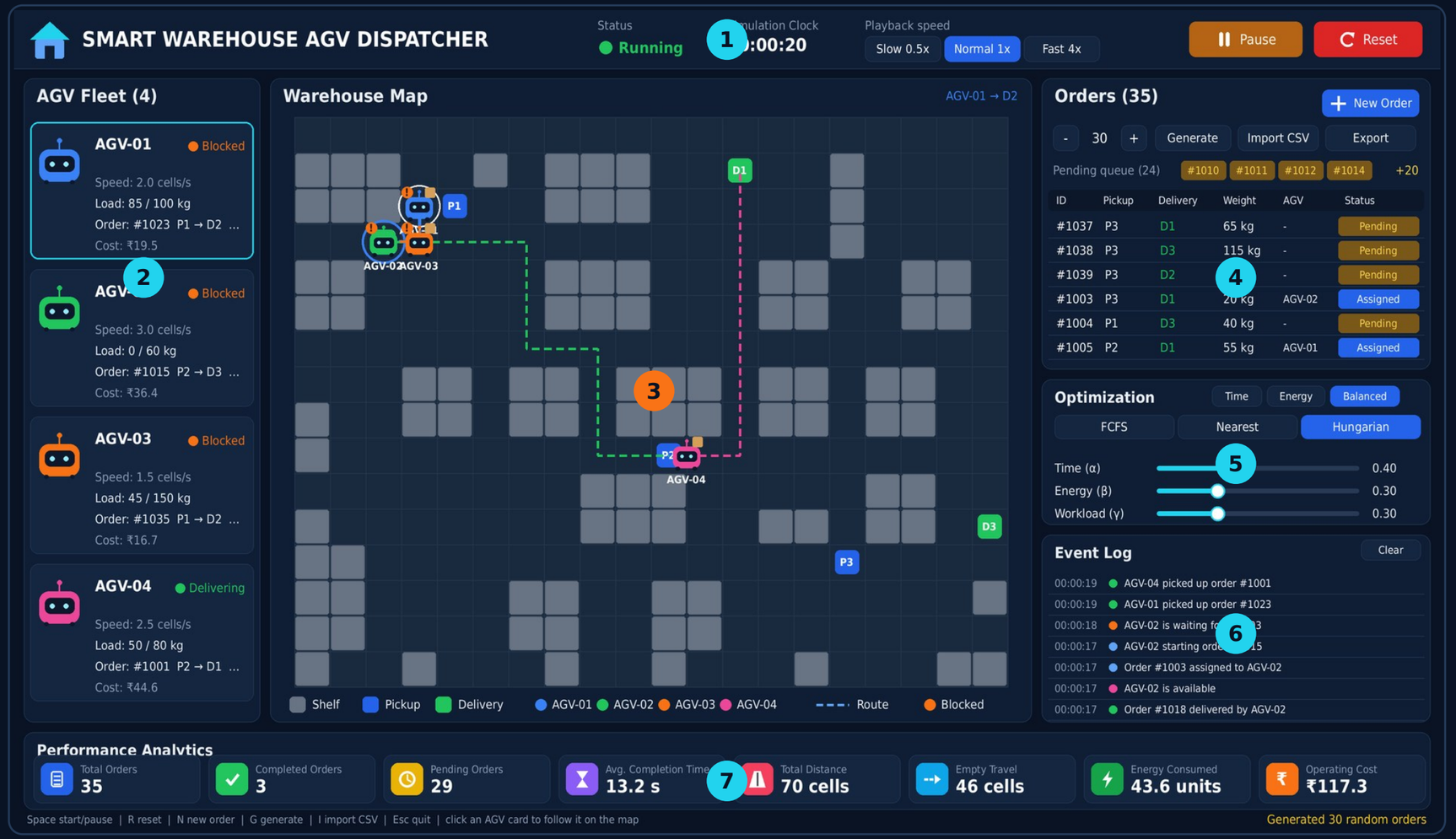
Idle AGVs begin assigned work using A* routes.

Key rule: started orders are never reassigned. Each AGV can hold one active order and one waiting order.

Dispatch is re-triggered only when the system state changes — keeping assignment responsive without interrupting deliveries.

11 INTERFACE

Dashboard and Visualization



1. **Top bar**
   status, clock, playback speed, Start / Pause / Reset
2. **AGV fleet**
   speed, load, order, status, cost; click to follow
3. **Warehouse map**
   robots, dashed routes, selected target, blocked marks
4. **Orders**
   table, pending queue, add / generate / import / export
5. **Optimization**
   strategy, presets, weight sliders
6. **Event log**
   assignments, pickups, deliveries, waits
7. **Analytics**
   8 live metrics from the simulation

Everything on screen is live simulation data from the project's own code.

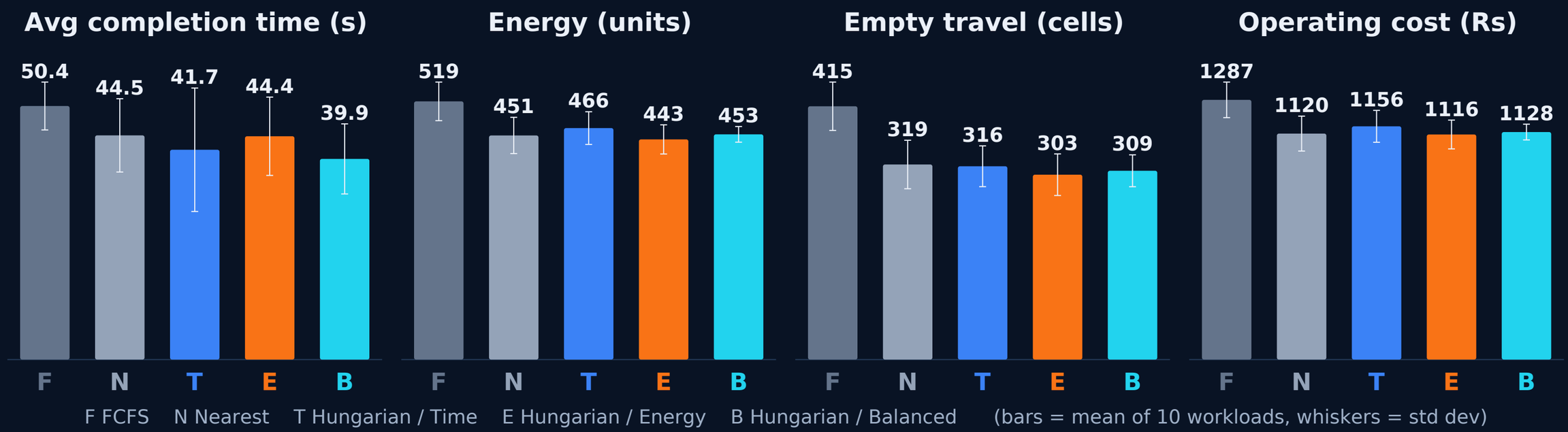
Performance Evaluation

50.4 s
FCFS
avg completion time

44.5 s
Nearest AGV
avg completion time

39.9 s
Hungarian (balanced)
avg completion time

443
Hungarian (energy)
energy units - lowest



Hungarian (balanced) was fastest on average in 10 workloads - a tendency, not a proof.

13 REFLECTION

What We Learned from This Project

Building the dispatcher helped us connect algorithms with a working, event-driven simulation.

Algorithm design

Implemented A* for shortest-path routing and the Hungarian algorithm for global task allocation.

Optimization trade-offs

Learned how time, energy and workload weights change which AGV receives an order.

Dynamic systems

Handled continuously arriving orders and re-planned assignments when the simulation state changed.

Multi-agent coordination

Used cell reservations, waiting, re-routing and back-off rules to reduce robot conflicts.

Software engineering

Separated the UI, simulation, algorithms and data models into independent modules.

Testing and evaluation

Compared FCFS, Nearest and Hungarian strategies using completion time, energy, empty travel and operating cost.

Main takeaway: a good warehouse dispatcher must balance routing, allocation, coordination and measurable performance.

14 SUMMARY

Conclusion, Real-World Use & Future Scope

WHAT THE PROJECT DELIVERS

A working prototype that assigns orders to heterogeneous AGVs, plans routes with A*, coordinates movement and visualizes operations in real time.

WHERE IT CAN BE USED

- E-commerce fulfilment and distribution centres
- Manufacturing plants moving parts between workstations
- Retail warehouses and inventory replenishment
- Hospital or campus logistics for internal deliveries

HOW IT COULD WORK IN A REAL WAREHOUSE

1 Map the floor

Add shelves, pickup and delivery points.

2 Register AGVs

Enter each robot’s speed, capacity and energy rate.

3 Feed orders

Connect a warehouse system or import an order CSV.

4 Dispatch & monitor

Allocate tasks, route robots and track live status.

PERFORMANCE — 10 WORKLOADS

Balanced Hungarian: 39.9 s average completion (vs 44.5 s Nearest and 50.4 s FCFS). Energy-focused Hungarian recorded the lowest energy use: 443 units. Balanced also reduced empty travel to 303 cells.

FUTURE SCOPE

Connect to live warehouse-management systems and sensors; test larger fleets and layouts; include congestion-aware travel-time estimates; and validate the dispatcher in a physical warehouse.